Sep 25, 2017 - First-Class Modules and Modular Implicits in OCaml (/modular-implicits.html)

Note: If you are reading this on github.com, please read it on my blog (http://tycon.github.io/modular-implicits.html), instead; it's much nicer.

I was pleasantly surprised to discover that OCaml has been supporting modules as first-class objects since v3.12 (2011). Intuition suggests that first-class modules should be expressive enough to simulate Haskell-style typeclasses in OCaml. Turns out this is the route taken by Leo White et al to introduce adhoc polymorphism via Modular Implicits (https://arxiv.org/abs/1512.01895) to OCaml. White et al's paper describes their principled approach in detail, and is definitely worth a read. In this blog, I present a series of OCaml examples involving (first-class) modules that gradually build up to modular implicits. The full code (with directions to run) is available here (https://github.com/gowthamk/Lambda/blob/master/okmij/modules.ml).

First lets recall the basics of modules and module types (signatures) in OCaml. Lets define a module type `T1`:

```
  module type T1 = sig
    type a
    val to_string: a -> string
    val eg: a
  end
```

Lets also define a couple of modules of type `T1`:

```
  module IntT1 : (T1 with type a = int) = struct
    type a = int
    let to_string = string_of_int
    let eg = 0
  end
(* module IntT1 : sig type a = int val to_string : a -> string end *)

  module BoolT1 : (T1 with type a = bool) = struct
    type a = bool
    let to_string = string_of_bool
    let eg = false
  end
```

Below is a functor that creates a module of type `T1`.

©2019 TYCON. Powered by Jekyll (http://jekyllrb.com), theme by Scott Emmons (https://github.com/scotte/jekyll-clean) under Creative Commons Attribution (http://creativecommons.org/licenses/by/4.0/)

```
module XT1(X:sig
              type t
              val to_string: t -> string
              val eg: t
           end) : (T1 with type a = X.t) = struct
  type a = X.t
  let to_string = X.to_string
  let eg = X.eg
end
```

The type of `XT1` is:

```
module XT1 :
  functor (X : sig type t val to_string : t -> string end) ->
      sig type a = X.t val to_string : a -> string end
```

OCaml has first class modules. A first-class module is created by “packing” a module with a signature. Done using the “module” keyword.

```
let i = (module IntT1: T1)
let b = (module BoolT1: T1)
```

It is possible to create a list of first-class modules, each encapsulating a different type:

```
let mod_list = [i; b]
```

However, i and b cannot be used as modules themselves; they need to be unpacked first. Unpacking is done using the “val” keyword:

```
module I = (val i : T1)
module B = (val b : T1)
```

Note that the information that type `a` in `I` is int is lost while packing it into a module of type `T1`. Thus `I.eg` is an abstract value of type `I.a`, as confirmed by utop (https://github.com/diml/utop):

```
    utop # I.eg;;
    - : I.a = <abstr>
```

Likewise, `B.eg` is an `<abstr>` value of type `B.a`.

Alternatively, we could’ve packed `IntT1` into a module of type `(T1 with type a = int)`, but that makes it impossible to create `[i;b]`.

It would have been clear by now that first-class modules are simply variables with existential types. We pack a module to introduce an existential type, and unpack it to eliminate the existential. Like the values of other types, values of existential types are first class. The `t2_of_t1` function shown below takes a value of type `module T1`, which is an existential type, and returns a value of type `module T2`, which is another existential type:

©2019 TYCON. Powered by Jekyll (http://jekyllrb.com), theme by Scott Emmons (https://github.com/scotte/jekyll-clean) under Creative Commons Attribution (http://creativecommons.org/licenses/by/4.0/)

```
module type T2 = sig
  include T1
  val print: a -> unit
end
  (** val t2_of_t1: (module T1) -> (module T2) *)
let t2_of_t1 (x:(module T1)) =
  let module X = (val x : T1) in
  let module Y = struct
                  include X
                  let print a = print_string @@ to_string a;;
                 end in
  let y = (module Y: T2) in
    y
```

Unpacking an existential (i.e., conversion from a first-class module to a normal module) is done automatically by OCaml during pattern matches, so `t2_of_t1` can be simplified as following:

```
let t2_of_t1 (module X: T1) =
  let module Y = struct
                  include X
                  let print a = print_string @@ to_string a;;
                 end in
  let y = (module Y: T2) in
    y
```

We can now construct modules at run-time:

```
module I2 = (val (t2_of_t1 i):T2)
```

This feature lets us choose between various implementations of a signature at run-time, depending on the execution parameters. Another big advantage of first-class modules is that it helps us achieve qualified/bounded/adhoc polymorphism, and lets us obtain Haskell-style typeclasses in ML. For instance, lets consider a SERIALIZABLE signature that characterizes all serializable types:

```
module type SERIALIZABLE = sig
  type t
  val to_string: t -> string
  val of_string: string -> t
end
(* For convenience *)
exception ConversionError
```

Abstract type `t` is supposed to be instantiated with any concrete serializable type. Base types such as int, float etc., are obviously serializable:

©2019 TYCON. Powered by Jekyll (http://jekyllrb.com), theme by Scott Emmons (https://github.com/scotte/jekyll-clean) under Creative Commons Attribution (http://creativecommons.org/licenses/by/4.0/)

```
module SInt : SERIALIZABLE with type t = int = struct
  type t = int
  let to_string = string_of_int
  let of_string = int_of_string
end
module SFloat : SERIALIZABLE with type t = float = struct
  type t = float
  let to_string = string_of_float
  let of_string = float_of_string
end
```

Polymorphic types like lists are serializable iff their type parameters are serializable:

```
module SList(A: SERIALIZABLE)
  : (SERIALIZABLE with type t = A.t list) = struct
  type t = A.t list
  let to_string l = "["^(String.concat ";" @@
                         List.map A.to_string l)^"]"
  let of_string s =
    let l = String.length s in
    let _ = if l>=2 && s.[0]='[' && s.[l-1]=']' then ()
            else raise ConversionError in
    let mid = String.sub s 1 (l-2) in
    let tokens = Str.split (Str.regexp ";") mid in
    let join p s = match p with "" -> s | _ -> p^";"^s in
    let els = List.rev @@ fst @@ List.fold_left
                (fun (els,p) s  ->
                  try
                    let el = A.of_string @@ join p s in
                      (el::els,"")
                  with ConversionError -> (els, join p s))
                ([],"") tokens in
      els
end
```

An aside: we implemented a serializable list as a *functor* above. An alternative approach is to implement it as a *function*. The second approach is more general than the first because functions can have polymorphic types but functors cannot (in OCaml).

```
(*
 * val mk_SList :
 *   (module SERIALIZABLE with type t = 'a) ->
 *     (module SERIALIZABLE with type t = 'a list)
 *)
let mk_SList (type a)
             (module A : SERIALIZABLE with type t = a) =
  let module SList = struct
        module S = SList(A)
        include S
      end in
    (module SList : SERIALIZABLE with type t = a list)
```

©2019 TYCON. Powered by Jekyll (http://jekyllrb.com), theme by Scott Emmons (https://github.com/scotte/jekyll-clean) under Creative Commons Attribution (http://creativecommons.org/licenses/by/4.0/)

We now have multiple `to_string` and `of_string` functions from SInt, SFloat, SList, and their compositions. Using first-class modules, we can write `to_string` and `of_string` wrappers for these functions that accept module parameters:

```
let to_string (type a)
              (module S: SERIALIZABLE with type t = a) = S.to_string
let of_string (type a)
              (module S: SERIALIZABLE with type t = a) = S.of_string
let "2" = to_string (module SInt) 2
let 3 = of_string (module SInt) "3"
let "[2;3]" = to_string (module SList(SInt)) [2;3];;
let [2;3] = of_string (module SList(SInt)) "[2;3]";;
let "[[2;3];[4;5]]" = to_string (module SList(SList(SInt))) [[2;3];[4;5]];;
let [[2;3];[4;5]] = of_string (module SList(SList(SInt))) "[[2;3];[4;5]]";;
```

Observe that in the above examples, single `to_string` / `of_string` function is being used for various SERIALIZABLE types. This is exactly the behavior that typeclasses offer in Haskell! There is, however, one significant difference. While GHC infers typeclasses automatically, here we have to explicitly pass the module parameters to `to_string` / `of_string` functions. This seems redundant, given that such module parameters can be inferred from the context. For example, when the `to_string` function is applied on `[[2;3];[4;5]]`, the only module parameter that lets this expression typecheck is `SList(SList(SInt))`, which must be inferable. This is infact the sort of reasoning performed by Modular Implicits (Leo White et al) OCaml extension, which lets us mark module parameters to a function as *implicit*, and later elide them when applying the function. This is demonstrated below:

We first mark the modules that serve as typeclass instances with the keyword `implicit`:

```
implicit module SInt : SERIALIZABLE with type t = int = struct
  type t = int
  let to_string = string_of_int
  let of_string = int_of_string
end

implicit module SFloat : SERIALIZABLE with type t = float = struct
  type t = float
  let to_string = string_of_float
  let of_string = float_of_string
end
```

Functors can also be marked implicit, as long as their arguments are implicit (marked via the curly braces).

©2019 TYCON. Powered by Jekyll (http://jekyllrb.com), theme by Scott Emmons (https://github.com/scotte/jekyll-clean) under Creative Commons Attribution (http://creativecommons.org/licenses/by/4.0/)

```
implicit module SList{A: SERIALIZABLE}
  : (SERIALIZABLE with type t = A.t list) = struct
  type t = A.t list
  let to_string l = "["^(String.concat ";" @@
                         List.map A.to_string l)^"]"
  let of_string s =
    let l = String.length s in
    let _ = if l>=2 && s.[0]='[' && s.[l-1]=']' then ()
            else raise ConversionError in
    let mid = String.sub s 1 (l-2) in
    let tokens = Str.split (Str.regexp ";") mid in
    let join p s = match p with "" -> s | _ -> p^";"^s in
    let els = List.rev @@ fst @@ List.fold_left
                (fun (els,p) s  ->
                  try
                    let el = A.of_string @@ join p s in
                      (el::els,"")
                  with ConversionError -> (els, join p s))
                ([],"") tokens in
      els
end
```

Lastly, we mark the module parameters of functions (for which adhoc polymorphism is intended) as implicit (again using curly braces):

```
let to_string (type a)
              {S: SERIALIZABLE with type t = a} = S.to_string
let of_string (type a)
              {S: SERIALIZABLE with type t = a} = S.of_string
```

Module parameters can now be elided:

```
let "2" = to_string 2
let 3 = of_string "3"
let "[2;3]" = to_string [2;3];;
let [2;3] = of_string "[2;3]";;
let "[[2;3];[4;5]]" = to_string [[2;3];[4;5]];;
let [[2;3];[4;5]] = of_string "[[2;3];[4;5]]";;
```

Observe that `to_string` and `of_string` do not take module parameters. The code behaves as if both the functions are part of a typeclass, for which int, float and list instances exist.

So, to conclude, **First-class modules + implicits = typeclasses in ML!**.

©2019 TYCON. Powered by Jekyll (http://jekyllrb.com), theme by Scott Emmons (https://github.com/scotte/jekyll-clean) under Creative Commons Attribution (http://creativecommons.org/licenses/by/4.0/)

0 Comments

Login

G

Start the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS

Name

3 Share

Best Newest Oldest

Be the first to comment.

Subscribe Privacy Do Not Sell My Data

Newer Post (/git-inconsistencies.html) Older Post (/effective-serializability.html)

©2019 TYCON. Powered by Jekyll (http://jekyllrb.com), theme by Scott Emmons (https://github.com/scotte/jekyll-clean) under Creative Commons Attribution (http://creativecommons.org/licenses/by/4.0/)